

COMPLETE LINE-BY-LINE EXPLANATION: cloud_testing.py

Unified Cloud Monitoring Script for AWS, Azure, GCP & Local

LINE-BY-LINE BREAKDOWN

Lines 1-2: SHEBANG & FILE HEADER

```
python

#!/usr/bin/env python3
"""
```

Line 1: `#!/usr/bin/env python3`

- **Purpose:** Shebang (hashbang) line
- **Explanation:** Tells the operating system to execute this file using Python 3
- **Usage:** Allows script to be run directly: `./cloud_testing.py` (instead of `python3 cloud_testing.py`)
- **How it works:** When you make script executable with `chmod +x cloud_testing.py`, the OS reads this line and knows to invoke Python3
- **Why `/usr/bin/env`:** Searches for Python3 in system PATH, making it portable across systems

Lines 2-26: DOCSTRING

```
python

"""
cloud_monitor.py
Unified monitoring script for Local, AWS, Azure and GCP hosts.
```

- **Purpose:** Module-level docstring
- **Explanation:** Provides documentation visible when you run `python -m pydoc cloud_testing.py` or inspect via `help()`
- **Contents:**
 - Script name and purpose
 - Features list (auto-detection, metric collection, alerting, logging)
 - Installation instructions (`pip install ...`)
 - Configuration notes (environment variables)
 - Required environment variables for each alert type

SECTION 1: IMPORTS

Lines 28-31: Standard Library Imports

```
python

import os
import time
import socket
import logging
import smtplib
import argparse
from email.mime.text import MIMEText
from logging.handlers import RotatingFileHandler
```

Line 28: `import os`

- **Purpose:** Access operating system functionality
- **Usage in script:**
 - `os.getenv()` - Read environment variables (config, secrets)
 - Example: `os.getenv("CPU_THRESHOLD", 1)` - Get CPU threshold or default to 1

Line 29: `import time`

- **Purpose:** Time-related operations
- **Usage in script:**
 - `time.time()` - Get current Unix timestamp (used for network rate calculation)
 - `time.sleep()` - Pause between monitoring cycles
 - Example: `time.sleep(10)` - Wait 10 seconds before next check

Line 30: `import socket`

- **Purpose:** Network communication and hostname resolution
- **Usage in script:**
 - `socket.gethostname()` - Get computer's hostname
 - Used for including hostname in alerts

Line 31: `import logging`

- **Purpose:** Log messages to file and console
- **Usage in script:**
 - Create logger object: `logger = logging.getLogger("cloud_monitor")`

- Log messages: `logger.info()`, `logger.error()`, `logger.warning()`

Line 32: `import smtplib`

- **Purpose:** Send emails via SMTP protocol
- **Usage in script:**
 - `smtplib.SMTP()` - Connect to email server (Gmail, Outlook, etc.)
 - `server.starttls()` - Enable encryption
 - `server.login()` - Authenticate with credentials
 - `server.send_message()` - Send email alert

Line 33: `import argparse`

- **Purpose:** Parse command-line arguments
- **Usage in script:**
 - `argparse.ArgumentParser()` - Define CLI options
 - Allows: `python cloud_testing.py --once` or `--interval 20`

Line 34-35: Specialized Imports

```
python

from email.mime.text import MIMEText
from logging.handlers import RotatingFileHandler
```

`from email.mime.text import MIMEText`

- **Purpose:** Create email message objects
- **Usage:** Construct properly formatted email with subject, sender, recipient
- **Example in script:**

```
python

msg = MIMEText(body)           # Create text email
msg["Subject"] = subject       # Add subject
msg["From"] = SMTP_USER       # Add sender
msg["To"] = ALERT_EMAIL_TO     # Add recipient
```

`from logging.handlers import RotatingFileHandler`

- **Purpose:** Log to rotating file (archive old logs automatically)
- **Benefits:**
 - Prevents log file from growing infinitely
 - Keeps last 3 backup files (backupCount=3)

- Max file size: 5MB before rotation
 - Example: `cloud_monitor.log` → `cloud_monitor.log.1` → `cloud_monitor.log.2`
-

Lines 37-42: Conditional Imports (Try-Except)

```
python

try:
    import psutil
except ImportError:
    psutil = None

try:
    import requests
except ImportError:
    requests = None
```

Purpose: Gracefully handle missing optional dependencies

Line 37-39: `try/except psutil`

- **What it does:** Attempts to import `psutil` (system monitoring library)
- **If successful:** `psutil` is available for CPU, memory, disk, network metrics
- **If fails:** Sets `psutil = None` so script doesn't crash
- **Why optional:** Script can run if psutil missing (just won't collect metrics)
- **Install:** `pip install psutil`

Line 41-43: `try/except requests`

- **What it does:** Attempts to import `requests` (HTTP library)
- **If successful:** Can send HTTP requests to metadata endpoints and webhooks
- **If fails:** Sets `requests = None` so script continues without it
- **Usage:** Cloud metadata detection, Slack/Teams alerts
- **Install:** `pip install requests`

Why try-except? Makes script robust. If user hasn't installed optional library, script still runs (just with reduced features)

SECTION 2: CONFIGURATION

Lines 50-54: Threshold Settings

```
python
```

```
CPU_THRESHOLD = float(os.getenv("CPU_THRESHOLD", 1))
RAM_THRESHOLD = float(os.getenv("RAM_THRESHOLD", 1))
DISK_THRESHOLD = float(os.getenv("DISK_THRESHOLD", 1))
NET_ERROR_THRESHOLD = int(os.getenv("NET_ERROR_THRESHOLD", 1))
```

Pattern: `os.getenv("ENV_VAR", default_value)`

- **Purpose:** Read configuration from environment variables
- **Example:** `CPU_THRESHOLD = os.getenv("CPU_THRESHOLD", 1)`
 - Looks for `CPU_THRESHOLD` environment variable
 - If not found, uses default value `1`
 - Security: No hardcoded credentials in source code

Line 50: `CPU_THRESHOLD`

- **Type:** `float` (decimal number)
- **Example:** `80.5` means alert if CPU > 80.5%
- **Usage:** Later compared against `metrics["cpu_percent"]`
- **Set via:** `export CPU_THRESHOLD=85` (Linux/Mac) or `set CPU_THRESHOLD=85` (Windows)

Line 51-52: RAM & DISK thresholds

- Same concept as CPU
- **Example:** Alert if RAM > 90% or Disk > 95%

Line 53: `NET_ERROR_THRESHOLD`

- **Type:** `int` (integer, not float)
- **Purpose:** Alert if total network errors exceed threshold
- **Calculated as:** `errin + errout` (inbound errors + outbound errors)
- **Example:** `10` means alert if total errors > 10

Line 54: `CHECK_INTERVAL`

- **Type:** `int` (seconds)
- **Purpose:** How often to check metrics
- **Default:** 10 seconds
- **Usage:** `time.sleep(CHECK_INTERVAL)` between monitoring cycles
- **Set via:** `export CHECK_INTERVAL=30` (check every 30 seconds)

Line 55: `METADATA_TIMEOUT`

- **Type:** `float` (seconds)

- **Purpose:** Maximum time to wait for cloud metadata response
 - **Default:** 1.5 seconds
 - **Usage:** When querying AWS/Azure/GCP metadata endpoints
 - **Why low?** If endpoint doesn't respond quickly, it's probably not that cloud provider
-

Lines 57-63: Email Configuration

```
python

SMTP_SERVER = os.getenv("SMTP_SERVER", "smtp.gmail.com")
SMTP_PORT = int(os.getenv("SMTP_PORT", 587))
SMTP_USER = os.getenv("SMTP_USER", "imrealaryanjain@gmail.com")
SMTP_PASSWORD = os.getenv("SMTP_PASSWORD", "szcb yzsv qrke rebo")
ALERT_EMAIL_TO = os.getenv("ALERT_EMAIL_TO", "jaind5611@gmail.com,aryanakmtdjain20
LOG_FILE = os.getenv("LOG_FILE", "cloud_monitor.log")
```

Line 57: `SMTP_SERVER`

- **Purpose:** Email server hostname
- **Default:** `smtp.gmail.com` (Gmail's server)
- **Other examples:** `smtp.office365.com` (Outlook), `smtp.sendgrid.net`
- **Type:** String

Line 58: `SMTP_PORT`

- **Purpose:** Port to connect to email server
- **Default:** `587` (TLS/StartTLS standard)
- **Type:** `int` (converted from string)
- **Other common ports:** 25 (unencrypted), 465 (SSL), 587 (TLS)

Line 59: `SMTP_USER`

- **Purpose:** Email account username/address
- **Type:** String
- **Note:** Must match `SMTP_SERVER` (Gmail account for Gmail server)

Line 60: `SMTP_PASSWORD`

- **Purpose:** Email account password
- **⚠ SECURITY WARNING:**
 - Never hardcode real password!
 - For Gmail: Use App Password (not regular password)

- Better: Set via environment variable: `export SMTP_PASSWORD="mypassword"`
- The shown password is intentionally obscured example

Line 61: `ALERT_EMAIL_TO`

- **Purpose:** Who receives alerts
- **Type:** Comma-separated string (can have multiple recipients)
- **Example:** `"user1@example.com,user2@example.com"`
- **Usage:** When breach detected, email goes to these addresses

Line 62: `LOG_FILE`

- **Purpose:** Where to write log file
 - **Default:** `cloud_monitor.log` (in current directory)
 - **Type:** String file path
 - **Example:** `"logs/cloud_monitor.log"` (logs folder)
-

Lines 64-65: Webhook URLs

```
python

SLACK_WEBHOOK_URL = os.getenv("SLACK_WEBHOOK_URL", "")
TEAMS_WEBHOOK_URL = os.getenv("TEAMS_WEBHOOK_URL", "")
```

Line 64: `SLACK_WEBHOOK_URL`

- **Purpose:** Send alerts to Slack channel
- **Type:** String (webhook URL)
- **Default:** `""` (empty = disabled)
- **Format:** `https://hooks.slack.com/services/XXXXX/XXXXX/XXXXX`
- **How to get:** Slack → Create App → Incoming Webhooks → Copy URL
- **Usage:** `send_slack()` function posts message to this URL

Line 65: `TEAMS_WEBHOOK_URL`

- **Purpose:** Send alerts to Microsoft Teams channel
 - **Type:** String (webhook URL)
 - **Default:** `""` (empty = disabled)
 - **Format:** `https://outlook.webhook.office.com/webhookb2/XXXXX/XXXXX/XXXXX`
 - **How to get:** Teams → Channel → Connectors → Incoming Webhook
 - **Usage:** `send_teams()` function posts formatted card to this URL
-

SECTION 3: LOGGING SETUP

Lines 75-83: Logger Configuration

```
python

logger = logging.getLogger("cloud_monitor")
logger.setLevel(logging.INFO)

_formatter = logging.Formatter("%(asctime)s %(levelname)s [%(name)s] %(message)s")

_file_handler = RotatingFileHandler(LOG_FILE, maxBytes=5 * 1024 * 1024, backupCount=5)
_file_handler.setFormatter(_formatter)
logger.addHandler(_file_handler)

_console_handler = logging.StreamHandler()
_console_handler.setFormatter(_formatter)
logger.addHandler(_console_handler)
```

Line 75: `logger = logging.getLogger("cloud_monitor")`

- **Purpose:** Create logger object named "cloud_monitor"
- **Explanation:** This logger is used throughout script for all log messages
- **Usage:** `logger.info()`, `logger.error()`, etc.

Line 76: `logger.setLevel(logging.INFO)`

- **Purpose:** Set minimum logging level
- **Levels (in order):** DEBUG < INFO < WARNING < ERROR < CRITICAL
- **This setting:** Only log INFO and above (skip DEBUG messages)
- **Example levels:**
 - `DEBUG`: Detailed info for diagnosing problems
 - `INFO`: General operational messages ✓ (enabled)
 - `WARNING`: Something unexpected
 - `ERROR`: Serious problem ✓ (enabled)
 - `CRITICAL`: System might fail ✓ (enabled)

Line 78: `_formatter = logging.Formatter(...)`

- **Purpose:** Define log message format
- **Format string breakdown:**

`%(asctime)s` → Timestamp (e.g., 2024-01-15 10:30:45,123)
`%(levelname)s` → Log level (INFO, ERROR, etc.)
`[(name)s]` → Logger name in brackets (e.g., [cloud_monitor])
`%(message)s` → Actual message text

- **Example output:** `2024-01-15 10:30:45,123 INFO [cloud_monitor] Detected environment: aws`

Lines 80-82: File Handler Setup

```
python

_file_handler = RotatingFileHandler(LOG_FILE, maxBytes=5 * 1024 * 1024, backupCount=3)
_file_handler.setFormatter(_formatter)
logger.addHandler(_file_handler)
```

Line 80: `RotatingFileHandler(...)`

- **Purpose:** Log to rotating file (archives when too large)
- **Parameters:**
 - `LOG_FILE`: Filename (e.g., "cloud_monitor.log")
 - `maxBytes=5 * 1024 * 1024`: Max file size = 5MB (5 million bytes)
 - `backupCount=3`: Keep 3 backup files
- **How it works:**
 - `cloud_monitor.log` → grows to 5MB
 - Rotate: `cloud_monitor.log` → `cloud_monitor.log.1`
 - New `cloud_monitor.log` created
 - After 4 rotations: Delete oldest backup (keep only 3)

Line 81: `_file_handler.setFormatter(_formatter)`

- **Purpose:** Apply logging format to file handler
- **Result:** File logs have timestamp, level, name, message

Line 82: `logger.addHandler(_file_handler)`

- **Purpose:** Register file handler with logger
- **Effect:** All `logger.info/error` calls now write to file

Lines 84-86: Console Handler Setup

```
python
```

```
_console_handler = logging.StreamHandler()
_console_handler.setFormatter(_formatter)
logger.addHandler(_console_handler)
```

Line 84: `logging.StreamHandler()`

- **Purpose:** Log to console/terminal (stdout)
- **No parameters:** Defaults to sys.stdout (console)
- **Result:** Users see logs while script runs

Lines 85-86: Same as file handler (format and register)

Result of logging setup:

- Logs appear in BOTH file and console
 - Organized, readable format
 - File auto-rotates when large
 - Easy troubleshooting
-

SECTION 4: ENVIRONMENT DETECTION

Lines 90-98: Cloud Metadata Detection Helper

```
python

def _metadata_reachable(url, headers=None):
    """Helper: Check if a metadata endpoint is reachable."""
    if requests is None:
        return False
    try:
        r = requests.get(url, headers=headers or {}, timeout=METADATA_TIMEOUT)
        return r.status_code == 200
    except Exception:
        return False
```

Line 90: `def _metadata_reachable(url, headers=None):`

- **Purpose:** Check if cloud metadata endpoint responds
- **Parameters:**
 - `url`: Metadata endpoint URL (e.g., "`http://169.254.169.254/...`")
 - `headers`: HTTP headers (e.g., auth tokens)
- **Return:** `True` if endpoint accessible, `False` otherwise

Line 91-92: `if requests is None: return False`

- **Purpose:** Graceful failure if requests library not installed
- **Logic:** Can't make HTTP request without requests library
- **Return:** `False` (not reachable) because we can't check

Lines 93-96: HTTP Request

```
python

try:
    r = requests.get(url, headers=headers or {}, timeout=METADATA_TIMEOUT)
    return r.status_code == 200
except Exception:
    return False
```

Line 93: `try:`

- **Purpose:** Catch network errors gracefully

Line 94: `r = requests.get(url, headers=headers or {}, timeout=METADATA_TIMEOUT)`

- `requests.get()`: Make HTTP GET request
- `url`: Target endpoint
- `headers=headers or {}`: Use provided headers, or empty dict if None
- `timeout=METADATA_TIMEOUT`: Wait max 1.5 seconds (don't hang)
- `r`: Response object

Line 95: `return r.status_code == 200`

- **Purpose:** Return True only if status code is 200 (OK)
- **Logic:** HTTP 200 = successful, endpoint is reachable
- **Other codes:** 404, 500, timeout → treated as unreachable

Line 97: `except Exception: return False`

- **Purpose:** Any error (network timeout, DNS failure, etc.)
- **Action:** Return `False` (endpoint not reachable)

Lines 100-128: Main Environment Detection

```
python
```

```
def detect_environment():  
    """Return one of: 'aws', 'azure', 'gcp', 'local'."""
```

Purpose: Auto-detect which cloud provider (or local)

How it works: Try metadata endpoints in order; first success wins

AWS Detection (Lines 102-110)

```
python  
  
if requests is not None:  
    try:  
        token_resp = requests.put(  
            "http://169.254.169.254/latest/api/token",  
            headers={"X-aws-ec2-metadata-token-ttl-seconds": "21600"},  
            timeout=METADATA_TIMEOUT,  
        )  
        if token_resp.status_code == 200:  
            token = token_resp.text  
            r = requests.get(  
                "http://169.254.169.254/latest/meta-data/instance-id",  
                headers={"X-aws-ec2-metadata-token": token},  
                timeout=METADATA_TIMEOUT,  
            )  
            if r.status_code == 200:  
                return "aws"  
    except Exception:  
        pass
```

AWS IMDSv2 (Secure token-based metadata):

Line 102: `if requests is not None:`

- **Purpose:** Only attempt if requests library available

Lines 103-108: Get AWS metadata token

```
python  
  
token_resp = requests.put(  
    "http://169.254.169.254/latest/api/token",  
    headers={"X-aws-ec2-metadata-token-ttl-seconds": "21600"},  
    timeout=METADATA_TIMEOUT,  
)
```

- **What it does:** Request security token from AWS metadata service

- **Endpoint:** `169.254.169.254` - Special AWS-only IP (doesn't route to internet)
- **Method:** PUT (IMDSv2 requirement, more secure than GET)
- **Header:** `X-aws-ec2-metadata-token-ttl-seconds: 21600` - Token valid for 6 hours
- **Only works on AWS:** Non-AWS systems can't reach this IP

Lines 109-117: Verify token works

```
python

if token_resp.status_code == 200:
    token = token_resp.text
    r = requests.get(
        "http://169.254.169.254/latest/meta-data/instance-id",
        headers={"X-aws-ec2-metadata-token": token},
        timeout=METADATA_TIMEOUT,
    )
    if r.status_code == 200:
        return "aws"
```

- **Line 109:** Check if token request succeeded
- **Line 110:** Extract token from response
- **Lines 111-117:** Use token to fetch instance ID
- **Line 116:** Include token in headers (IMDSv2 requirement)
- **Line 118:** If this succeeds, definitely AWS EC2 instance

Line 119: `except Exception: pass`

- **Purpose:** If anything fails (timeout, network error), continue to next check
- **Note:** `pass` means do nothing, try next cloud provider

Azure Detection (Line 121)

```
python

if _metadata_reachable(
    "http://169.254.169.254/metadata/instance?api-version=2021-02-01",
    headers={"Metadata": "true"},
):
    return "azure"
```

How it works:

- **Endpoint:** Azure metadata endpoint (different path than AWS)
- **Header:** `Metadata: true` (required by Azure)
- **If reachable:** Return "azure"

- **Why this works:** Azure uses same IP but different path and headers

GCP Detection (Line 124)

```
python

if _metadata_reachable(
    "http://metadata.google.internal/computeMetadata/v1/instance/id",
    headers={"Metadata-Flavor": "Google"},
):
    return "gcp"
```

How it works:

- **Endpoint:** GCP uses different DNS name and path
- **Header:** `(Metadata-Flavor: Google)` (GCP requirement)
- **If reachable:** Return "gcp"

Fallback to Local (Line 127)

```
python

return "local"
```

Purpose: If no cloud metadata endpoints respond, assume running locally

Summary of detection logic:

Try AWS → Try Azure → Try GCP → Default to Local

SECTION 5: METRIC COLLECTION

Lines 138-141: Global Variable for Network Tracking

```
python

_last_net_counters = {"time": None, "sent": 0, "recv": 0}
```

Purpose: Track previous network statistics to calculate rate of change

Dict contents:

- `(time)`: Timestamp of last check (Unix time)
- `(sent)`: Total bytes sent (cumulative)
- `(recv)`: Total bytes received (cumulative)

- Formula: $(\text{new_total} - \text{old_total}) / \text{time_elapsed}$
 - Example: If sent bytes went from 1000→1500 in 10 seconds = 50 bytes/sec = 0.05 KB/sec
-

Lines 144-171: Main Metrics Collection Function

python

```
def collect_metrics():  
    """Collect CPU / RAM / Disk / Network metrics. Returns a dict."""  
    if psutil is None:  
        logger.error("psutil is not installed; cannot collect system metrics.")  
        return {}
```

Purpose: Gather all system metrics

Line 145-147: Check psutil installed

- **If not:** Log error and return empty dict
- **Why:** Script can't function without psutil

python

```
cpu = psutil.cpu_percent(interval=1)  
mem = psutil.virtual_memory()  
disk = psutil.disk_usage("/")  
net = psutil.net_io_counters()
```

Line 149: `cpu = psutil.cpu_percent(interval=1)`

- **Purpose:** Get CPU usage percentage
- **Parameter:** `interval=1` - Measure over 1 second
- **Return:** Float (0-100, e.g., 45.3 = 45.3%)
- **What it measures:** Average across all cores

Line 150: `mem = psutil.virtual_memory()`

- **Purpose:** Get RAM usage info
- **Return:** Object with: `.total`, `.available`, `.percent`, etc.
- **Example:** `mem.percent` = 65.4 (65.4% of RAM used)

Line 151: `disk = psutil.disk_usage("/")`

- **Purpose:** Get disk usage of root filesystem

- **Parameter:** `("/")` - Root directory (or `"C:\"` on Windows)
- **Return:** Object with: `.total`, `.used`, `.free`, `.percent`

Line 152: `net = psutil.net_io_counters()`

- **Purpose:** Get total network statistics since boot
- **Return:** Object with: `.bytes_sent`, `.bytes_recv`, `.errin`, `.errout`

Lines 154-162: Calculate Network Rate

python

```
now = time.time()
sent_rate = recv_rate = 0.0
if _last_net_counters["time"] is not None:
    elapsed = max(now - _last_net_counters["time"], 1e-6)
    sent_rate = (net.bytes_sent - _last_net_counters["sent"]) / elapsed / 1024
    recv_rate = (net.bytes_recv - _last_net_counters["recv"]) / elapsed / 1024
    _last_net_counters.update(time=now, sent=net.bytes_sent, recv=net.bytes_recv)
```

Line 154: `now = time.time()`

- **Purpose:** Get current Unix timestamp
- **Value:** Seconds since Jan 1, 1970 (e.g., 1705324245.123)

Line 155: `sent_rate = recv_rate = 0.0`

- **Purpose:** Initialize rates to 0
- **Type:** Float (decimal)
- **Why:** If first run, no previous data to calculate rate

Line 156: `if _last_net_counters["time"] is not None:`

- **Purpose:** Check if we have previous measurement
- **First run:** `time` is None, so skip calculation
- **Subsequent runs:** Calculate rate

Line 157: `elapsed = max(now - _last_net_counters["time"], 1e-6)`

- **Purpose:** Time elapsed since last check
- **Calculation:** Current time - previous time (e.g., 10.5 seconds)
- `max(..., 1e-6)`: Ensure not zero (prevents division by zero)
- `1e-6`: 0.000001 seconds (one millionth second, fallback minimum)

Lines 158-159: Network rate calculation

Sent rate example:

- Previous: sent_bytes = 1,000,000
- Current: sent_bytes = 1,102,400
- Elapsed: 10 seconds
- Calculation:

```
(1,102,400 - 1,000,000) / 10 / 1024  
= 102,400 / 10 / 1024  
= 10,240 / 1024  
= 10 KB/sec
```

/ 1024: Convert bytes to kilobytes

- 1024 bytes = 1 KB

Line 161: `_last_net_counters.update(...)`

- **Purpose:** Save current values for next cycle
- **Updates dict with:**
 - `time`: now (current timestamp)
 - `sent`: net.bytes_sent (current total)
 - `recv`: net.bytes_recv (current total)

Lines 163-173: Return Metrics Dictionary

python

```
metrics = {  
    "hostname": socket.gethostname(),  
    "cpu_percent": cpu,  
    "ram_percent": mem.percent,  
    "disk_percent": disk.percent,  
    "net_sent_kbps": round(sent_rate, 2),  
    "net_recv_kbps": round(recv_rate, 2),  
    "net_errin": net.errin,  
    "net_errout": net.errout,  
}  
return metrics
```

Purpose: Return all metrics in organized dict

Each key explained:

- `"hostname"`: Computer name (e.g., "web-server-01")
 - `"cpu_percent"`: CPU % (e.g., 45.3)
 - `"ram_percent"`: RAM % (e.g., 78.9)
 - `"disk_percent"`: Disk % (e.g., 62.1)
 - `"net_sent_kbps"`: KB/sec sent (e.g., 123.45)
 - `round(..., 2)`: Round to 2 decimal places
 - `"net_recv_kbps"`: KB/sec received (e.g., 456.78)
 - `"net_errin"`: Network errors inbound (cumulative, e.g., 5)
 - `"net_errout"`: Network errors outbound (cumulative, e.g., 3)
-

SECTION 6: ALERT EVALUATION

Lines 176-194: Check Thresholds

```
python

def evaluate_alerts(metrics):
    """Return a list of human-readable alert strings for any breached threshold."""
    alerts = []
    if not metrics:
        return alerts
```

Purpose: Determine which thresholds were exceeded

Line 178: `alerts = []`

- **Purpose:** Initialize empty list for alert messages

Lines 179-180: Empty metrics check

- **If no metrics:** Return empty list (no alerts possible)

Lines 181-189: Check each threshold

```
python
```

```

if metrics["cpu_percent"] > CPU_THRESHOLD:
    alerts.append(f"CPU high: {metrics['cpu_percent']}% (threshold {CPU_THRESHH
if metrics["ram_percent"] > RAM_THRESHOLD:
    alerts.append(f"RAM high: {metrics['ram_percent']}% (threshold {RAM_THRESHH
if metrics["disk_percent"] > DISK_THRESHOLD:
    alerts.append(f"Disk high: {metrics['disk_percent']}% (threshold {DISK_THR

```

Example - CPU check:

- **If:** Current CPU 85% > Threshold 80%
- **Then:** Add string to alerts list: `"CPU high: 85% (threshold 80%)"`
- **Purpose:** Log or send in alert message

Same logic for RAM and Disk

Lines 191-194: Network errors check

```

python

net_errors = metrics["net_errin"] + metrics["net_errout"]
if net_errors > NET_ERROR_THRESHOLD:
    alerts.append(f"Network errors high: {net_errors} (threshold {NET_ERROR_TH
return alerts

```

Line 191: `net_errors = metrics["net_errin"] + metrics["net_errout"]`

- **Purpose:** Total network errors
- **Example:** 5 inbound + 3 outbound = 8 total errors

Lines 192-194: Check against threshold and add to list

Return: List of alert strings

- **Empty list:** No issues
- **Non-empty:** Issues detected, e.g., `["CPU high: 85%", "RAM high: 95%"]`

SECTION 7: CLOUD-SPECIFIC EXTRAS

Lines 200-204: AWS Extra Info

```

python

```

```
def extra_aws_info():
    """Fetch extra AWS-specific info (best-effort)."""
    try:
        import boto3
        ec2 = boto3.client("ec2")
        res = ec2.describe_instances()
        count = sum(len(r["Instances"]) for r in res["Reservations"])
        return f"AWS EC2 instances visible to these credentials: {count}"
    except Exception as e:
        logger.warning(f"AWS extra info unavailable: {e}")
        return None
```

Purpose: Provide additional AWS info (not critical)

Line 201: `import boto3`

- **Purpose:** Import AWS SDK within function
- **Note:** Only imported if function called (lazy import)
- **Requirement:** `pip install boto3`

Line 202: `ec2 = boto3.client("ec2")`

- **Purpose:** Create EC2 service client
- **Uses:** Credentials from environment/IAM role

Line 203: `res = ec2.describe_instances()`

- **Purpose:** Fetch all EC2 instances
- **Return:** Response with reservations and instances

Line 204: `count = sum(len(r["Instances"]) for r in res["Reservations"])`

- **Purpose:** Count total instances
- **Logic:** Sum instance counts across all reservations
- **Example:** 3 reservations with [2, 5, 1] instances = 8 total

Line 205: `return f"AWS EC2 instances..."`

- **Purpose:** Return readable message with count

Lines 206-207: Exception handling

python

```
except Exception as e:
    logger.warning(f"AWS extra info unavailable: {e}")
    return None
```

- **Purpose:** If error occurs (no credentials, boto3 missing, etc.)
 - **Action:** Log warning and return None (non-fatal)
-

Lines 210-215: Azure Extra Info

```
python

def extra_azure_info():
    """Azure-specific info (placeholder)."""
    try:
        from azure.identity import DefaultAzureCredential # noqa: F401
        return "Azure detected. Configure azure-mgmt-compute calls here for VM-level"
    except Exception as e:
        logger.warning(f"Azure extra info unavailable: {e}")
        return None
```

Similar pattern to AWS:

- Try import Azure SDK
 - Return message if successful
 - Log warning if not available
 - This is a placeholder (real implementation would query Azure VMs)
-

Lines 218-223: GCP Extra Info

```
python

def extra_gcp_info():
    """GCP-specific info (placeholder)."""
    try:
        from google.cloud import compute_v1 # noqa: F401
        return "GCP detected. Configure google-cloud-compute calls here for instance"
    except Exception as e:
        logger.warning(f"GCP extra info unavailable: {e}")
        return None
```

SECTION 8: ALERTING FUNCTIONS

Lines 229-243: Email Alert

python

```
def send_email(subject, body):  
    """Send email alert via SMTP."""  
    if not (SMTP_SERVER and SMTP_USER and SMTP_PASSWORD and ALERT_EMAIL_TO):  
        logger.debug("Email not configured; skipping email alert.")  
        return
```

Purpose: Send threshold breach via email

Line 230: Configuration check

- **If any required field missing:** Skip email
- **Fields needed:** Server, User, Password, Recipient
- **Reason:** Incomplete config = can't send email

python

```
try:  
    msg = MIMEText(body)  
    msg["Subject"] = subject  
    msg["From"] = SMTP_USER  
    msg["To"] = ALERT_EMAIL_TO
```

Lines 233-236: Create email message

- **MIMEText(body):** Create email with text body
- **msg["Subject"]:** Add subject line
- **msg["From"]:** Sender address
- **msg["To"]:** Recipient(s)

python

```
with smtplib.SMTP(SMTP_SERVER, SMTP_PORT, timeout=10) as s:  
    s.starttls()  
    s.login(SMTP_USER, SMTP_PASSWORD)  
    s.send_message(msg)  
logger.info("Email alert sent.")
```

Lines 237-241: Send email

- `smtplib.SMTP(...)`: Connect to email server
 - `SMTP_SERVER`: Server hostname (smtp.gmail.com)
 - `SMTP_PORT`: Port (587)
 - `timeout=10`: Don't wait more than 10 seconds
 - `with ... as s:` Automatically close connection when done
- `s.starttls()`: Upgrade to encrypted connection
- `s.login(...)`: Authenticate with email credentials
- `s.send_message(msg)`: Send message
- **Line 241**: Log success

python

```
except Exception as e:  
    logger.error(f"Failed to send email alert: {e}")
```

Line 242-243: Catch and log any errors

Lines 246-258: Slack Alert

python

```
def send_slack(subject, body):  
    """Send alert to Slack webhook."""  
    if not SLACK_WEBHOOK_URL:  
        logger.debug("Slack not configured; skipping Slack alert.")  
        return  
    if requests is None:  
        logger.error("requests library not installed; cannot send Slack alert.")  
        return
```

Purpose: Post alert message to Slack channel

Lines 247-249: Check configuration

- **If no webhook URL:** Skip (not configured)
- **If requests missing:** Skip (can't make HTTP request)

python

```
try:
    payload = {"text": f"*{subject}*\n{body}"}
    r = requests.post(SLACK_WEBHOOK_URL, json=payload, timeout=10)
    r.raise_for_status()
    logger.info("Slack alert sent.")
except Exception as e:
    logger.error(f"Failed to send Slack alert: {e}")
```

Line 253: `payload = {"text": ...}`

- **Purpose:** Message format Slack expects
- `*{subject}*`: Bold text (Slack markdown)
- `\n{body}`: New line, then body text

Line 254: `requests.post(...)`

- `SLACK_WEBHOOK_URL`: Webhook URL (from Slack)
- `json=payload`: Send payload as JSON
- `timeout=10`: Don't hang more than 10 seconds
- `r`: Response object

Line 255: `r.raise_for_status()`

- **Purpose:** Raise exception if HTTP error (4xx, 5xx)
- **Example:** 404 Not Found → raises exception → caught in except

Lines 261-279: Teams Alert

```
python
```

```
def send_teams(subject, body):
    """Send alert to MS Teams webhook."""
    if not TEAMS_WEBHOOK_URL:
        logger.debug("Teams not configured; skipping Teams alert.")
        return
    if requests is None:
        logger.error("requests library not installed; cannot send Teams alert.")
        return
    try:
        payload = {
            "@type": "MessageCard",
            "@context": "http://schema.org/extensions",
            "summary": subject,
            "themeColor": "FF0000",
            "title": subject,
            "text": body.replace("\n", "\n\n"),
        }
    }
```

Purpose: Post alert to Teams channel

Similar structure to Slack, but different payload format

Line 271-277: MessageCard format

- `@type: "MessageCard"`: Teams card type (formatted message)
- `@context`: Schema reference
- `summary`: Brief message (for notifications)
- `themeColor: "FF0000"`: Red color (indicates alert/error)
- `title`: Large heading text
- `text`: Body text
 - `.replace("\n", "\n\n")`: Double newlines for better formatting

python

```
r = requests.post(TEAMS_WEBHOOK_URL, json=payload, timeout=10)
r.raise_for_status()
logger.info("Teams alert sent.")
except Exception as e:
    logger.error(f"Failed to send Teams alert: {e}")
```

Same pattern as Slack (post, check status, log result)

Lines 282-293: Dispatch Alerts

python

```
def dispatch_alerts(env, metrics, alerts):  
    """Send alerts through all configured channels."""  
    if not alerts:  
        return  
    subject = f"[{env.upper()}] Resource Alert on {metrics.get('hostname', 'unknown host')}"  
    body = "\n".join(alerts)  
    logger.warning(f"Threshold breach detected: {body}")  
    send_email(subject, body)  
    send_slack(subject, body)  
    send_teams(subject, body)
```

Purpose: Send alerts through all channels

Line 283-284: Early return if no alerts

Line 285: Build subject line

- `env.upper()`: Uppercase environment (aws → AWS)
- `metrics.get('hostname', ...)`: Get hostname or default to "unknown host"
- **Example:** "[AWS] Resource Alert on web-server-01"

Line 286: Join alert strings

- `"\n".join(alerts)`: Connect multiple alerts with newlines
- **Example:** "CPU high: 85%\nRAM high: 95%"

Line 287: Log warning message

Lines 288-290: Send to all channels

- **Email** (if configured)
- **Slack** (if configured)
- **Teams** (if configured)

SECTION 9: MAIN MONITORING LOOP

Lines 296-320: Single Monitoring Cycle

python

```
def run_once(env):
    """Execute one monitoring cycle: collect metrics, evaluate, alert."""
    metrics = collect_metrics()
    if metrics:
        logger.info(
            f"[{env.upper()}] host={metrics['hostname']} "
            f"CPU={metrics['cpu_percent']}% RAM={metrics['ram_percent']}% "
            f"DISK={metrics['disk_percent']}% "
            f"NET_SENT={metrics['net_sent_kbps']}KB/s NET_RECV={metrics['net_recv_']
            f"NET_ERR={metrics['net_errin'] + metrics['net_errout']}"
        )
```

Purpose: One complete monitoring iteration

Line 297: `metrics = collect_metrics()`

- **Purpose:** Gather current system stats
- **Return:** Dict with CPU, RAM, Disk, Network

Lines 298-308: Log metrics

- **Check:** If metrics exist (not empty)
- **Format:** Single line with all key metrics
- **Example output:**

```
[AWS] host=server-1 CPU=45.3% RAM=78.9% DISK=62.1% NET_SENT=123.45KB/s
NET_RECV=456.78KB/s NET_ERR=8
```

python

```
alerts = evaluate_alerts(metrics)
dispatch_alerts(env, metrics, alerts)
```

Line 309: Check which thresholds exceeded

- **Return:** List of alert messages

Line 310: Send alerts to all channels

python

```

extra_info = {"aws": extra_aws_info, "azure": extra_azure_info, "gcp": extra_g
if extra_info:
    info = extra_info()
    if info:
        logger.info(info)

```

Line 311: Get function for current environment

- **Dict:** Maps env name to function
- **.get(env):** Get function if env matches, otherwise None
- **Example:** If env="aws", extra_info=extra_aws_info

Lines 312-314: Call function and log result

- **if extra_info:** Check if function exists
- **info = extra_info():** Call function
- **if info:** Log if not None

Lines 323-344: Main Function & CLI

```

python

def main():
    """Main entry point."""
    parser = argparse.ArgumentParser(description="Unified Local/AWS/Azure/GCP reso
    parser.add_argument("--once", action="store_true", help="Run a single check an
    parser.add_argument(
        "--interval", type=int, default=CHECK_INTERVAL, help="Seconds between chec
    )
    args = parser.parse_args()

```

Purpose: Parse command-line arguments and start monitoring

Lines 324-325: Create argument parser

- **ArgumentParser():** Create CLI argument handler
- **description:** Help text shown with **--help**

Lines 326-327: **--once** flag

- **action="store_true":** Boolean flag (default: False)
- **Usage:** `python cloud_testing.py --once` (run once, don't loop)

- **Help text:** Shown with `-h`

Lines 328-331: `--interval` option

- `type=int`: Expect integer value
- `default=CHECK_INTERVAL`: Default (from config, e.g., 10)
- `help=...`: Description shown with `-h`
- **Usage:** `python cloud_testing.py --interval 30` (check every 30 sec)
- `%(default)s`: Placeholder replaced with default value in help

Line 332: `args = parser.parse_args()`

- **Purpose:** Parse command-line arguments
- **Result:** `args.once` (bool) and `args.interval` (int)

```
python
```

```
if psutil is None:
    logger.error("psutil is required. Install it with: pip install psutil")
    return
```

Lines 334-336: Check psutil dependency

- **If missing:** Log error and exit
- **Purpose:** Can't run without psutil

```
python
```

```
env = detect_environment()
logger.info(f"Detected environment: {env.upper()}")
```

Lines 338-339: Detect and log environment

- **Line 338:** Run environment detection
- **Line 339:** Log result (AWS, Azure, GCP, or Local)

```
python
```

```
if args.once:
    run_once(env)
    return
```

Lines 341-342: Single run mode

- **If `--once` flag:** Run one cycle and exit

python

```
while True:
    try:
        run_once(env)
    except Exception as e:
        logger.error(f"Unhandled error during monitoring cycle: {e}")
    logger.info(f"Sleeping {args.interval} seconds...")
    time.sleep(args.interval)
```

Lines 344-350: Continuous monitoring loop

- **while True:** Infinite loop
- **Line 345-347:** Try run_once, catch any errors
- **Line 349:** Log sleep message
- **Line 350:** Pause before next cycle
- **Flow:**

```
Collect metrics
↓
Evaluate alerts
↓
Send alerts (if any)
↓
Sleep 10 seconds
↓
Repeat
```

Lines 353-355: Script Entry Point

python

```
if __name__ == "__main__":
    main()
```

Purpose: Execute main() only when script run directly

if __name__ == "__main__":

- **True when:** Script run directly (`python cloud_testing.py`)
- **False when:** Script imported as module (`from cloud_testing import collect_metrics`)
- **Benefit:** Prevents main() executing on import

main(): Call main function

USAGE EXAMPLES

Example 1: Basic usage (detect environment, monitor continuously)

```
bash

export CPU_THRESHOLD=85
export RAM_THRESHOLD=90
export SLACK_WEBHOOK_URL="https://hooks.slack.com/services/..."

python cloud_testing.py
```

What happens:

1. Detect environment (AWS, Azure, GCP, or Local)
2. Check CPU < 85% and RAM < 90% every 10 seconds
3. If breach, send Slack alert

Example 2: One-time check

```
bash

python cloud_testing.py --once
```

Output: Single metrics check, then exit

Example 3: Custom interval

```
bash

python cloud_testing.py --interval 30
```

Effect: Check every 30 seconds (instead of default 10)

Example 4: With all alerts

```
bash
```

```
export SMTP_SERVER=smtp.gmail.com
export SMTP_USER=monitor@example.com
export SMTP_PASSWORD=app_password_here
export ALERT_EMAIL_TO=admin@example.com
export SLACK_WEBHOOK_URL=https://hooks.slack.com/...
export TEAMS_WEBHOOK_URL=https://outlook.webhook.office.com/...

python cloud_testing.py --interval 60
```

Effect: Check every 60 seconds, send alerts via Email + Slack + Teams if thresholds exceeded

SUMMARY TABLE

Component	Purpose	Example
Environment Detection	Identify cloud provider	Returns: "aws", "azure", "gcp", or "local"
Metric Collection	Gather system stats	CPU 45.3%, RAM 78.9%, Disk 62.1%
Alert Evaluation	Check thresholds	CPU > 80% → add to alerts
Alerting	Send notifications	Email, Slack, Teams
Logging	Record activity	File + Console output
CLI Arguments	User options	--once, --interval N

KEY TAKEAWAYS

- ✔ **Portable:** Works on AWS, Azure, GCP, or local machine
- ✔ **Configurable:** All settings via environment variables
- ✔ **Secure:** No hardcoded credentials
- ✔ **Resilient:** Graceful handling of missing libraries
- ✔ **Alerting:** Multiple channels (Email, Slack, Teams)
- ✔ **Logging:** File rotation + console output
- ✔ **Cloud-aware:** Detects cloud provider automatically